CSC 212: Data Structures and Abstractions

14: Binary Search Trees (part 1)

Prof. Marco Alvarez

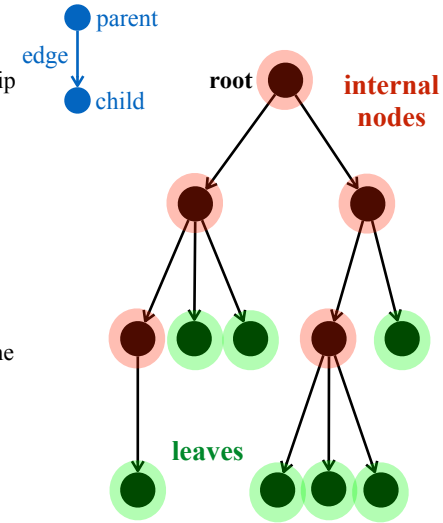
Department of Computer Science and Statistics
University of Rhode Island

Spring 2026

Trees

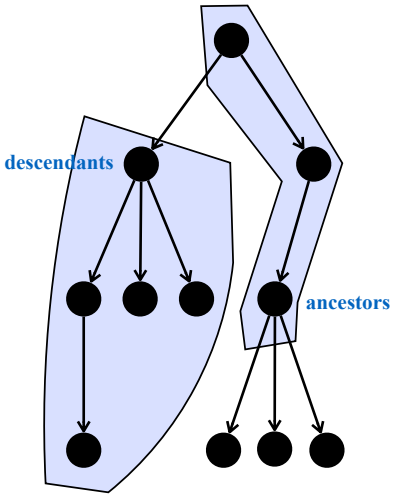
Trees

- Definition
  - a tree T is a set of **nodes** storing elements in a **parent-child** relationship with the following properties:
    - if T is nonempty, it has a special node, called the root of T, that has no parent
    - each node v of T different from the root has a unique parent node w; every node with parent w is a child of w
- Other relationships
  - two nodes that are children of the same parent are **siblings**
  - a node v is **external** if v has no children (a.k.a. leaves)
  - a node v is **internal** if it has one or more children



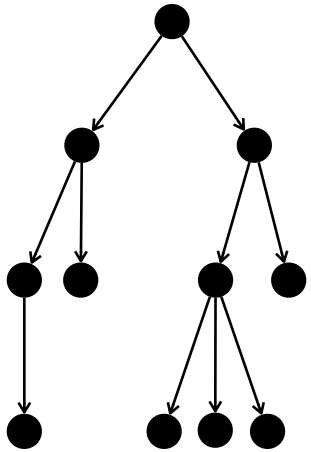
Trees

- Other relationships
  - a node u is an **ancestor** of a node v if $u = v$ or u is an ancestor of the parent of v
  - a node v is a **descendant** of a node u if u is an ancestor of v
  - the **subtree** of T rooted at a node v is the tree consisting of all the descendants of v in T (including v itself)
  - an **edge** of tree T is a pair of nodes (u, v) such that u is the parent of v, or vice versa
  - a **path** of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge



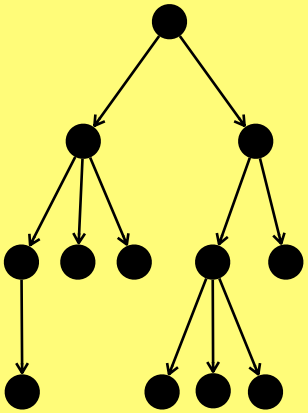
Depth and height

- The **length of a path** is the number of edges in the path
- The **depth** of a node v is the length of the path from the root node to v
- The **height** of a node v is the length of the path from v to its farthest descendant
- The **height of the tree** is the height of the root



Practice

- Label all nodes with height and depth
  - indicate the height and the depth of the tree

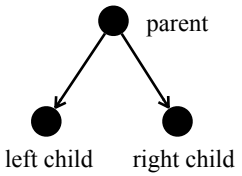


k-ary trees

- **k-ary tree**
  - a tree in which every internal node has at most k children
- **Full k-ary tree**
  - a tree in which every internal node has exactly k children
- **Complete k-ary tree**
  - given h representing the height of the tree, all levels 0 through $h - 1$ are completely filled, and level h has all nodes as far left as possible
- **Perfect k-ary tree**
  - all internal nodes have exactly k children and all leaves are at the same depth

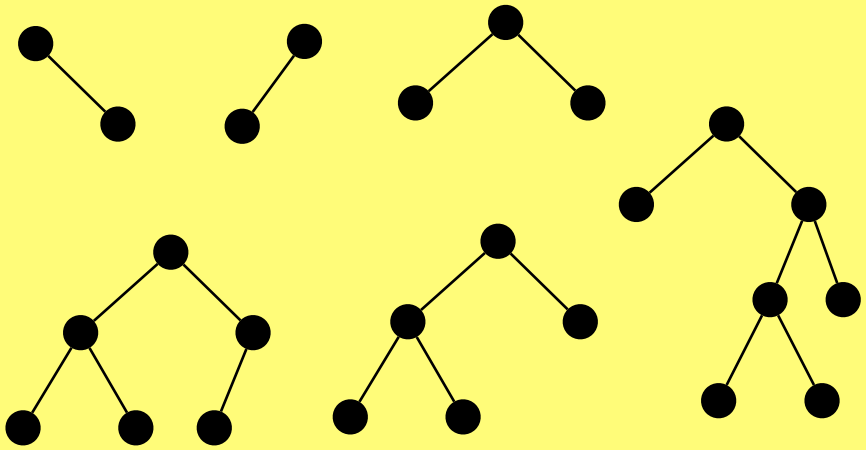
Binary trees

- Definition
  - a special case of a k-ary tree, where $k = 2$
- Properties
  - every node has at most 2 children
  - children are distinguished as left child and right child
  - the subtrees rooted at these children are called the left subtree and right subtree



Practice

- Mark the following binary trees (k=2) as full/complete/ perfect

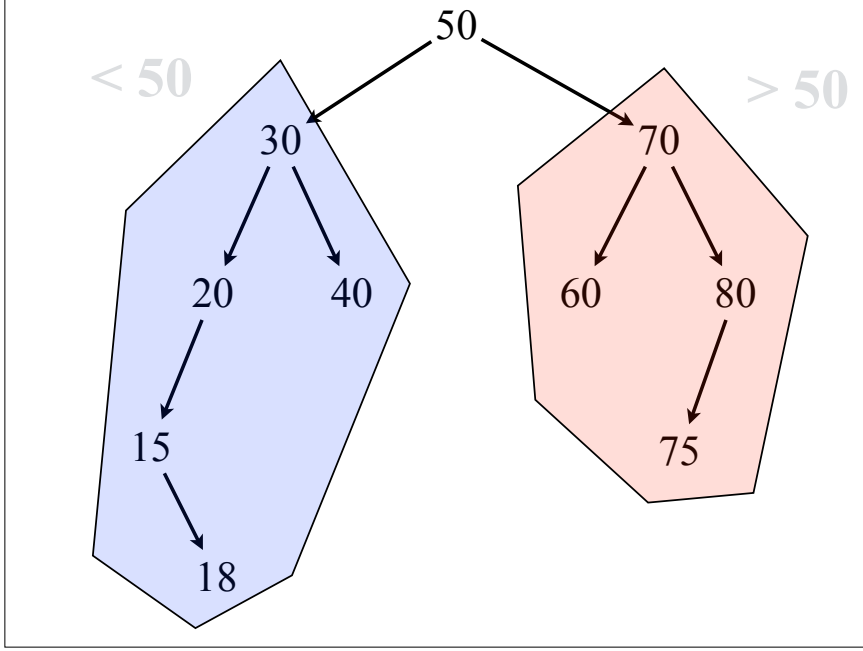


Binary search trees

Binary search tree

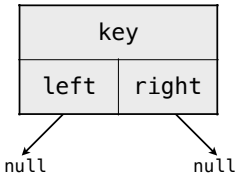
- A binary search tree (BST) is a binary tree that satisfies the **BST property** (also called symmetric order)
- **Symmetric order**
  - each node x in a BST has a key denoted by $key(x)$
  - for all nodes y in the left subtree of x, $key(y) < key(x)$ **
  - for all nodes y in the right subtree of x, $key(y) > key(x)$ **
  - this property must hold for every node in the tree

(**) assume that the keys of a BST are pairwise distinct



Representing a node

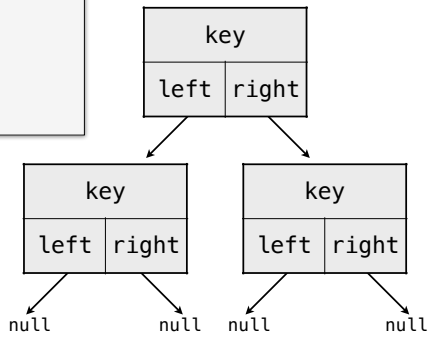
```
template <typename T>
class Node {
    private:
        T key;
        Node<T> *left, *right;
        friend class BST<T>;

    public:
        Node(const T& value) {
            key = value;
            left = right = nullptr;
        }
};
```



The implementation of a **BST node** requires a structure that can accommodate connections to two child nodes

Representing a binary search tree

```
template <typename T>
class BST {
    private:
        Node<T> *root;
        size_t size;
        bool search_helper(Node<T>* p,const T& target) const;
        Node<T>* insert_helper(Node<T>* p, const T& value);
        Node<T>* remove_helper(Node<T>* p, const T& value);

    public:
        BST() : root(nullptr), size(0) {}
        ~BST() { clear(); }
        size_t getSize() const { return size; }
        bool empty() const { return size == 0; }

        void insert(const T& value);
        void remove(const T& value);
        bool search(const T& value) const;
        void clear();
};
```



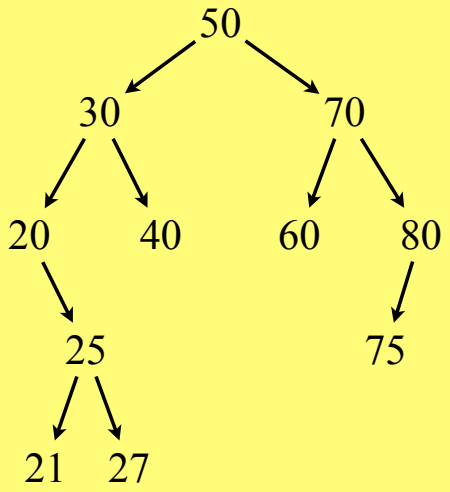
Operations: search and insert

Search

- Algorithm
  - ✓ start at root node
  - ✓ if the search key matches the current node's key return found
  - ✓ if search key is greater than current node's key
    - recursively search the right subtree
  - ✓ if search key is less than current node's key
    - recursively search the left subtree
  - ✓ stop when the current node is nullptr (key not found)
- Time complexity
  - ✓ $O(h)$, where h is the height of the tree

Practice

- Search the following keys:
  - 25, 77, 18, 40, 75



Recursive search

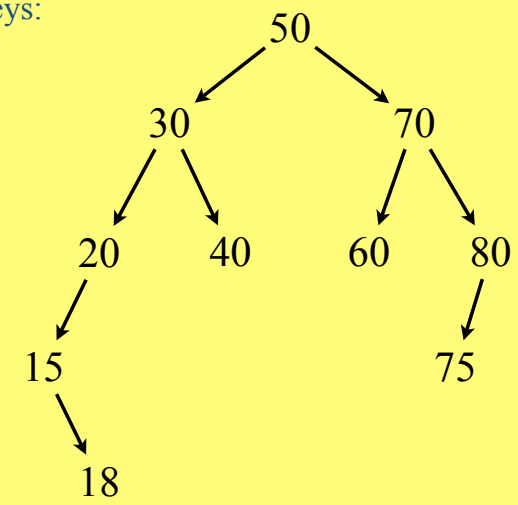
```
template <typename T>
bool BST<T>::search(const T& value) const {
    return search_helper(root, value);
}


template <typename T>
bool BST<T>::search_helper(Node<T>* p, const T& target) const {
    if ( !p ) {
        return false;
    }
    if (target < p->key) {
        return search_helper(p->left, target);
    } else if (target > p->key) {
        return search_helper(p->right, target);
    } else {
        return true;
    }
}
```


Insert

- Algorithm
  - if tree is empty, create a new node as the root and done
  - otherwise, start at the root node:
    - compare the key to insert with the current node's key
      - if equal, the key already exists — done
      - if the new key is less than the current node's key
        - if left child is empty, create new node as left child — done
        - otherwise, move to the left child and repeat
      - if the new key is greater than the current node's key
        - if right child is empty, create new node as right child — done
        - otherwise, move to the right child and repeat
- Time complexity
  - $O(h)$, where h is the height of the tree

Practice

- Insert the following keys:
  - 65, 27, 90, 11, 51



Recursive insert

```
template <typename T>
void BST<T>::insert(const T& value) {
    root = insert_helper(root, value);
    ++size;
}


template <typename T>
Node<T>* BST<T>::insert_helper(Node<T>* p, const T& value) {
    if ( !p ) {
        return new Node<T>(value);
    }
    if (value < p->key) {
        p->left = insert_helper(p->left, value);
    } else if (value > p->key) {
        p->right = insert_helper(p->right, value);
    }
    return p;
}
```

Find the bug

Repeated keys

- Assumption
  - ✓ the BST contains unique keys, no duplicate keys are allowed in the standard implementation
- Handling duplicate keys
  - ✓ **strategy 1**: ignore duplicates
    - if key is in the tree, do nothing and return
  - ✓ **strategy 2**: frequency counter
    - add a counter/frequency field to each node
    - increment the counter when a duplicate key is inserted
  - ✓ **strategy 3**: update associated value
    - if the BST is used as a map/dictionary (key-value pairs)
    - replace the old value with the new value for the duplicate key